SAECOIN — NODE OPERATIONS GUIDE

Document: Node_Operations.md
Version: 1.0
Project: SAECOIN
Ticker: SAC (Provisional)
Status: Draft / Planning
Consensus: Proof of Work
Transaction Model: UTXO

---

1. PURPOSE

This document defines the operational procedures for running and maintaining a SAECOIN Full Node.

It covers:

- Node installation
- Initial configuration
- Node startup and shutdown
- Blockchain synchronization
- Peer management
- Database management
- Network connectivity
- Monitoring
- Backup and recovery
- Troubleshooting
- Security

The objective is to make it possible for ordinary users to operate independent SAECOIN nodes.

---

2. WHAT IS A FULL NODE?

A Full Node is software that independently validates blockchain data according to SAECOIN consensus rules.

Its responsibilities include:

- Downloading blockchain blocks
- Validating block headers
- Verifying Proof of Work
- Validating transactions
- Maintaining UTXO state

- Enforcing monetary rules
- Maintaining peer connections
- Sharing valid blockchain data
- Rejecting invalid blocks and transactions

A Full Node must not blindly trust another node's reported balance or blockchain state.

---

3. NODE TYPES

SAECOIN may support the following node configurations.

Node Type| Purpose
Full Node| Independently validates the blockchain
Mining Node| Participates in block production
Archive Node| Retains complete historical blockchain data
Pruned Node| Deletes eligible old block data while preserving required validation state
Development Node| Runs a local development network
Testnet Node| Participates in testnet

The exact support for pruning and archival behavior must be finalized during implementation.

A mining node should use the same consensus validation rules as a regular Full Node.

---

4. SYSTEM REQUIREMENTS

Final requirements will depend on network size and blockchain growth.

Initial planning recommendations:

Resource| Development Recommendation
Operating System| Windows 11 or supported Linux
RAM| 16 GB preferred for development
Storage| SSD
Network| Stable broadband
CPU| Modern multi-core processor
Internet| Reliable connection

These are development recommendations, not guaranteed production minimums.

Actual requirements must be measured through benchmarking.

---

5. NODE INSTALLATION

The final SAECOIN release package should contain:

```
SAECOIN-NODE/
   saecoin-node.exe
   saecoin-cli.exe
   README.md
   LICENSE
   config-example/
```

Installation procedure:

1. Download the official release.
2. Verify the SHA-256 checksum.
3. Extract the package.
4. Create the appropriate data directory.
5. Prepare the node configuration.
6. Select the intended network.
7. Start the node.
8. Verify the node status.

Only official, verified release packages should be used for production nodes.

---

6. NODE CONFIGURATION

The node should support a configuration file.

Proposed example:

```
[network]
network = "testnet"

[node]
data_dir = "./data"
max_peers = 32

[p2p]
enabled = true
listen = true
port = 38444

[api]
enabled = false
bind = "127.0.0.1"
port = 38445
```

```
[logging]
level = "info"
```

This is an illustrative configuration.

All values are provisional and must match the finalized Network Parameters Specification.

The public API should remain disabled unless explicitly required.

---

7. NODE STARTUP

The future node application should support a command similar to:

```
.\saecoin-node.exe --network testnet
```

Expected startup sequence:

1. Load configuration.
2. Validate configuration.
3. Initialize logging.
4. Open the correct data directory.
5. Verify network identity.
6. Initialize the database.
7. Load the current chain state.
8. Initialize the P2P subsystem.
9. Discover or connect to peers.
10. Begin blockchain synchronization.
11. Report node status.

A node must stop startup if it detects an incompatible network or corrupted critical configuration.

The command shown above is a proposed interface, not an implemented command.

---

8. BLOCKCHAIN SYNCHRONIZATION

When a new node starts, it must synchronize with the network.

Proposed process:

1. Connect to eligible peers.
2. Exchange protocol information.
3. Verify network identity.
4. Obtain peer chain information.

5. Request required block data.
6. Validate received blocks.
7. Apply valid blocks to the local chain.
8. Update UTXO state.
9. Continue until synchronized.

The node must validate blockchain data independently.

A peer's reported chain height must not be accepted as proof that its chain is valid.

---

9. PEER MANAGEMENT

The node should manage inbound and outbound connections.

Required capabilities:

- Peer discovery
- Peer connection
- Peer disconnection
- Peer health monitoring
- Duplicate connection handling
- Connection timeouts
- Invalid-message rejection
- Peer scoring
- Temporary banning of abusive peers

Peer-management limits must be configurable.

Peer data should not be treated as trusted information.

---

10. NODE COMMAND-LINE INTERFACE

The future CLI should support operational commands.

Proposed commands:

saecoin-cli getinfo
saecoin-cli getblockchaininfo
saecoin-cli getnetworkinfo
saecoin-cli getpeerinfo
saecoin-cli getblock <block_hash>
saecoin-cli getmempoolinfo
saecoin-cli stop

These commands are proposed interfaces.

Their exact names, arguments, output formats, and permissions must be finalized during API and CLI implementation.

---

11. NODE MONITORING

Node operators should be able to monitor:

- Current block height
- Best block hash
- Synchronization status
- Peer count
- Incoming connections
- Outgoing connections
- Network traffic
- Database size
- Disk usage
- Memory usage
- CPU usage
- Mempool size
- Recent errors

Monitoring should not expose private wallet information.

A node's public status endpoint should reveal only information intended for public access.

---

12. LOGGING

Node software should maintain operational logs.

Suggested logging levels:

Level| Purpose
ERROR| Serious failures
WARN| Potential problems
INFO| Normal operational events
DEBUG| Development diagnostics
TRACE| Detailed troubleshooting

Production nodes should not use TRACE logging by default.

Logs must never contain:

- Private keys
- Recovery phrases
- Passwords
- Wallet secrets
- Sensitive authentication tokens

Log rotation and storage limits should be implemented.

---

13. DATABASE MANAGEMENT

The node database stores blockchain information and current chain state.

Proposed logical data areas:

```
data/
    blocks/
    chainstate/
    indexes/
    peers/
    logs/
```

The actual database engine and storage layout remain subject to final architecture decisions.

Database requirements:

- Atomic state updates
- Crash recovery
- Corruption detection
- Consistent block indexing
- UTXO state consistency
- Safe shutdown
- Versioned database migrations

Database files must not be manually modified by ordinary node operators.

---

14. BACKUP POLICY

Node backup procedures should distinguish between:

14.1 Public Blockchain Data

Most blockchain data can potentially be downloaded again from the network.

14.2 Node Configuration

Back up:

- Network configuration
- Custom node settings
- Operator-specific configuration

14.3 Wallet Data

Wallet data requires stronger protection.

Wallet backup procedures must follow Wallet Security Design.

Private keys and recovery phrases must never be included in ordinary node backups unless securely encrypted and explicitly intended.

---

15. SAFE NODE SHUTDOWN

The node must support a controlled shutdown process.

Proposed command:

saecoin-cli stop

Expected behavior:

1. Stop accepting new work.
2. Finish or safely cancel active operations.
3. Close peer connections.
4. Persist required database state.
5. Flush pending writes.
6. Close database handles.
7. Exit cleanly.

Users should avoid forcibly terminating a node during database writes.

Crash recovery must still be tested and supported.

---

16. NODE SECURITY

Node operators should:

- Download software from official releases.

- Verify release checksums.
- Keep software updated.
- Protect configuration files.
- Restrict unnecessary network ports.
- Disable unused APIs.
- Avoid exposing administrative interfaces publicly.
- Use firewall rules.
- Monitor disk usage.
- Review logs for unusual activity.

A node must treat incoming peer messages as untrusted.

A node must never allow a remote peer to execute arbitrary local commands.

---

17. NODE UPGRADES

Before upgrading:

1. Read the release notes.
2. Verify the new package.
3. Review compatibility requirements.
4. Back up important configuration.
5. Stop the existing node safely.
6. Install the new release.
7. Restart the node.
8. Verify synchronization.
9. Check logs.
10. Confirm normal operation.

Consensus-breaking upgrades require additional coordination and must follow the official protocol upgrade policy.

---

18. TROUBLESHOOTING

Problem| Possible Cause| Recommended Action
Node cannot start| Invalid configuration| Review configuration
No peers connected| Network or firewall issue| Check connectivity
Synchronization is slow| Low bandwidth or few peers| Review peer health
Database cannot open| File permissions or corruption| Inspect logs and recovery procedure
High RAM usage| Large workload or memory leak| Monitor and report
High disk usage| Blockchain growth| Check storage capacity
Repeated peer disconnections| Network instability or protocol mismatch| Review peer logs
Wrong network detected| Incorrect configuration| Select the intended network

Recovery must never bypass consensus validation.

---

19. NODE PERFORMANCE TESTING

The node should be tested under:

- Initial blockchain synchronization
- Long-running operation
- High transaction volume
- Large peer counts
- Sudden network disconnection
- Database restart
- Crash recovery
- Chain reorganization
- High block propagation load

Performance results should be documented.

The project must publish realistic hardware recommendations based on measured results.

---

20. NODE OPERATOR RESPONSIBILITIES

Node operators should:

1. Keep software updated.
2. Maintain sufficient storage.
3. Monitor node health.
4. Protect their systems.
5. Avoid sharing sensitive information.
6. Verify official releases.
7. Report security vulnerabilities responsibly.
8. Maintain reliable network connectivity.
9. Follow documented upgrade procedures.
10. Report operational problems with useful logs that contain no secrets.

Running a Full Node does not automatically provide mining rewards.

Mining participation requires the appropriate mining functionality and valid block production.

---

21. FUTURE NODE FEATURES

Potential future features:

- Graphical node dashboard
- Automatic software update notifications
- Advanced peer monitoring
- Pruning support
- Archive-node mode
- Remote monitoring
- Resource usage alerts
- Automated backup assistance
- Node health reports

These features are optional and should not delay the first functional Full Node.

---

22. FINAL OPERATIONAL PRINCIPLES

1. Nodes validate independently.
2. Network data is untrusted until verified.
3. Mainnet and testnet data remain separate.
4. Database updates must be crash-safe.
5. APIs remain restricted by default.
6. Private wallet information must remain protected.
7. Shutdown and recovery must be reliable.
8. Upgrades must follow documented procedures.
9. Node resource requirements must be benchmarked.
10. Consensus rules must never be bypassed for operational convenience.

DOCUMENT STATUS

Document: Node_Operations.md
Version: 1.0
Project: SAECOIN
Node Type: Full Node
Database: Not Finalized
P2P Protocol: Not Finalized
CLI Interface: Proposed
Network Ports: Provisional
Implementation Status: Not Started